Tentamen TEN1 HI1029 2023-06-07

Sista ordinarie insläpp: 8.00 – dörren stängs.
Tentavakterna delar ut konton. Logga in i tentasystemet så fort som möjligt.
När alla loggat in (förhoppningsvis 8.15) börjar tentamen och pågår i 4h 45min (tex 8.15-13.00).
8.30 Insläpp för försenade.

Extra skrivtid 50%: 7h 10min (tex 8.15-15.25) *(endast 1h extra skrivtid för A eller B)*
Skrivvakter: Anteckna start- och inlämningstid. Jour: Nicklas Brandefelt 087909738

Hjälpmedel: papper (tomma), penna, mus

Logga in med tentamenskontot ni får av skrivvakten. Logga inte ut förrän ni är klara med hela tentan.

Skapa ett nytt projekt i IntelliJ: **Project Name:** TEN1, **Project Location:** H

Alla: Skapa ett package per uppgift och kalla dessa uppgift1o2, uppgift3, uppgift4,…, uppgift7. Kopiera hjälpfiler från P/HI1029/ till rätt package.

NYTT: Svaren på teorifrågorna (fråga ett och två) skall besvaras i en textfil döpt till teori.txt som ni lägger i package uppgift1o2.

Även om ni inte löser hela uppgiften så kan ni få delpoäng men ni måste se till att koden kompilerar och kan köras. Både output och koden bedöms och det är viktigt att denna är välskriven och uppfyller uppgiftens instruktioner. Om ett program är ineffektivt kan detta ge poängavdrag. Det är meningen att man skall använda de tekniker vi lärt oss där detta är fördelaktigt. Välj bra namn för variabler, metoder och klasser. Se till att programmet har en bra struktur. Ni behöver på denna tentamen inte kommentera.

Hjälpfiler hittar ni i P/HI1029/ och Java API i P/Java/api/index.html.

Del 1 (15p): För godkänt krävs minst 1 poäng på teorifrågorna samt 9 poäng totalt. Om man får minst en poäng på teorifrågorna får man betyg enligt: 9p – E, 10p – D, 12p – C. Om man inte får minst en poäng på teorifrågorna men minst 8,5 poäng totalt får man Fx.

Del2: Bedöms bara om man får betyg C på del 1 och bedöms då med A, B eller C. Observera att om du har extra skrivtid får du bara använda 1 extra timme för att kunna få betyg A eller B.

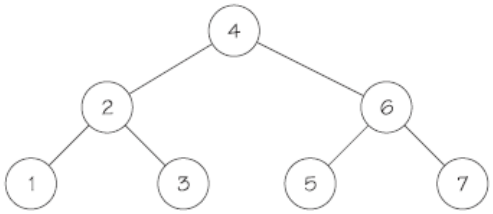
Examinator och rättande lärare: Nicklas Brandefelt

Del 1

1. Ange och motivera ordo för operationen add(E) på en väl-implementerad lista med en array som inre datastruktur (add(E) lägger ett element sist i listan). Diskutera ordo i olika fall och diskutera vad det genomsnittliga värdet blir i en bra implementation och varför. (1p) *Svaret skall lämnas i en textfil, teori.txt i H.*

2. Alla klasser i java ärver metoderna equals och hashCode från klassen Object. Förklara hur metoderna bör hänga ihop och varför. (1p) *Svaret skall lämnas i en textfil, teori.txt i H.*

3. Givet en förenklad version av vårt binära sökträd (P/HI1029/uppgift3) skriv en metod som returnerar en String med alla medlemmar i trädet enligt bredden först. Trädet nedan ska då returnera: ”4, 2, 6, 1, 3, 5, 7, ”. Observera att metoden ska klara godtyckligt träd. Om t.ex. trädet är tomt ska metoden returnera en tom sträng. (3p)



4. Givet en matris av positiva heltal och en startposition på första raden. Hitta den väg som ger den högsta summan om man summerar alla besökta platser på väg ner till valfri plats på sista raden. Vid varje förflyttning måste man gå till någon av de tre närmsta platserna på nästa rad (förutsatt att man inte är vid kanten då man bara kan gå till de två närmsta platserna på raden nedanför). Om du t.ex. startar på femman i matrisen nedan kan du gå till tvåan, sexan eller trean i raden nedanför. Från tvåan i raden nedanför däremot kan du bara gå till trean eller femman. I gult kan du se vägen med högst summa (29) om du börjar på femman.

1	5	4	7	8	3	1
2	6	3	4	7	1	5
3	5	1	9	1	7	1
1	3	1	4	5	8	2
3	1	8	3	1	2	1

a) Skriv en metod som tar en heltalsmatris (n×m) och en startkolumn (0,…,m-1) och returnerar den högsta summan enligt ovan. Använd en rekursiv djupet först lösning. Ett enkelt skal hittar du i P/HI1029/uppgift4. (3p)

b) Effektivisera din lösning med hjälp av dynamisk programmering. Du kan göra en separat metod om du vill. (2p)

5. Skriv en metod som sorterar en array av heltal med hjälp av mergesort algoritmen. Du får inte använda Java's sorteringsalgoritm utan skall skriva din egen. Utgå ifrån skalet du hittar i P/HI1029/uppgift5. (3p)

   Tips: Mergesort bygger på att man delar upp array:en i två delar och sorterar varje del separat. Sedan sammanfogar man dessa två sorterade array:er. Detta gör man rekursivt.

6. Givet en array av heltal hitta ett par av heltal som blir en specifik summa.

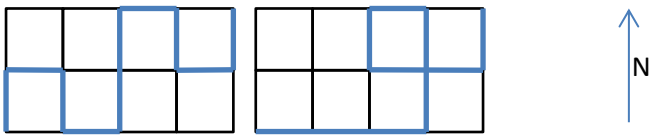
   Ex. För array:en {3,4,1,7,9,5,3,6} och summan 13: kan man hitta (4, 9) eller (6,7).

   Skriv en metod som tar en array av heltal och en summa och returnerar en array med ett av paren som har korrekt summa. Om det finns flera par behöver den hitta endast en. Om det saknas par med korrekt summa ska den returnera null. Metoden ska lösa uppgiften i O(n) och för full poäng vara så effektiv som möjligt. I katalogen P/HI1029/uppgift6 hittar du ett enkelt skal. (2p)

   Tips: Använd en Hashtabell.

Del 2

7. I den här uppgiften ska du beräkna antalet möjliga vägar från övre högra till nedre vänstra hörnet i en rektangulär stad. Observera att man i varje korsning får gå åt vilket håll man vill. Man får passera samma korsning två gånger men inte gå på samma gata två gånger. Nedan ser du två exempel på möjliga vägar genom en stad med 2 × 4 kvarter.



   Observera att den sista bilden kan motsvara två olika vägar genom staden. Antingen S, V, V, N, Ö, S, S, V, V, V eller S, V, N, V, S, Ö, S, V, V, V. Du ska skriva en metod som tar antalet kvarter i väst-östlig riktning och antalet kvarter i nord-sydlig riktning och returnerar antalet möjliga vägar. I staden på bilden ovan (2 × 4 kvarter) finns 335 möjliga vägar från övre högra hörnet till nedre vänstra hörnet. I en stad med 2 × 2 kvarter finns 16 möjliga vägar. I en stad med 1 × 3 kvarter finns 8 möjliga vägar.

   Tips: Använd en tvådimensionell array där varannan plats är en korsning och varannan plats är en gata som datastruktur.